CO3-AT1 – Question 39

Name: Talari Vishnuvardhan

Register No: 192572091

Subject: Design and Analysis of Algorithms

Faculty: Dr. F. Mary Harin Fernandez

Question 39

Title

Analyze Binary Search Performance for Datasets with Frequent Updates Requiring Re-sorting. Record execution time for update and search operations. Interpret trade-offs between maintenance cost and search efficiency. Justify appropriate use cases.

1. Aim

To analyze the performance of the Binary Search algorithm on datasets with frequent updates requiring re-sorting, measure the execution time of update and search operations, and study the trade-off between maintenance cost and search efficiency.

2. Objective

- To implement the Binary Search algorithm.
- To create a sorted dataset.
- To insert new elements into the dataset.
- To re-sort the dataset after every update.
- To measure update execution time.
- To measure search execution time.
- To compare maintenance cost and search efficiency.
- To identify suitable applications of Binary Search.

3. Introduction

Binary Search is a fast searching algorithm that works only on sorted data. It repeatedly divides the search space into two halves until the target element is found. Its time complexity is O(log n), making it much faster than Linear Search for large datasets.

When new elements are inserted, the dataset becomes unsorted. Therefore, it must be sorted again before Binary Search can be used. Re-sorting increases the maintenance cost. This experiment measures both update time and search time to understand the balance between maintaining sorted data and achieving fast searches.

4. Theory

Binary Search compares the search element with the middle element of a sorted array.

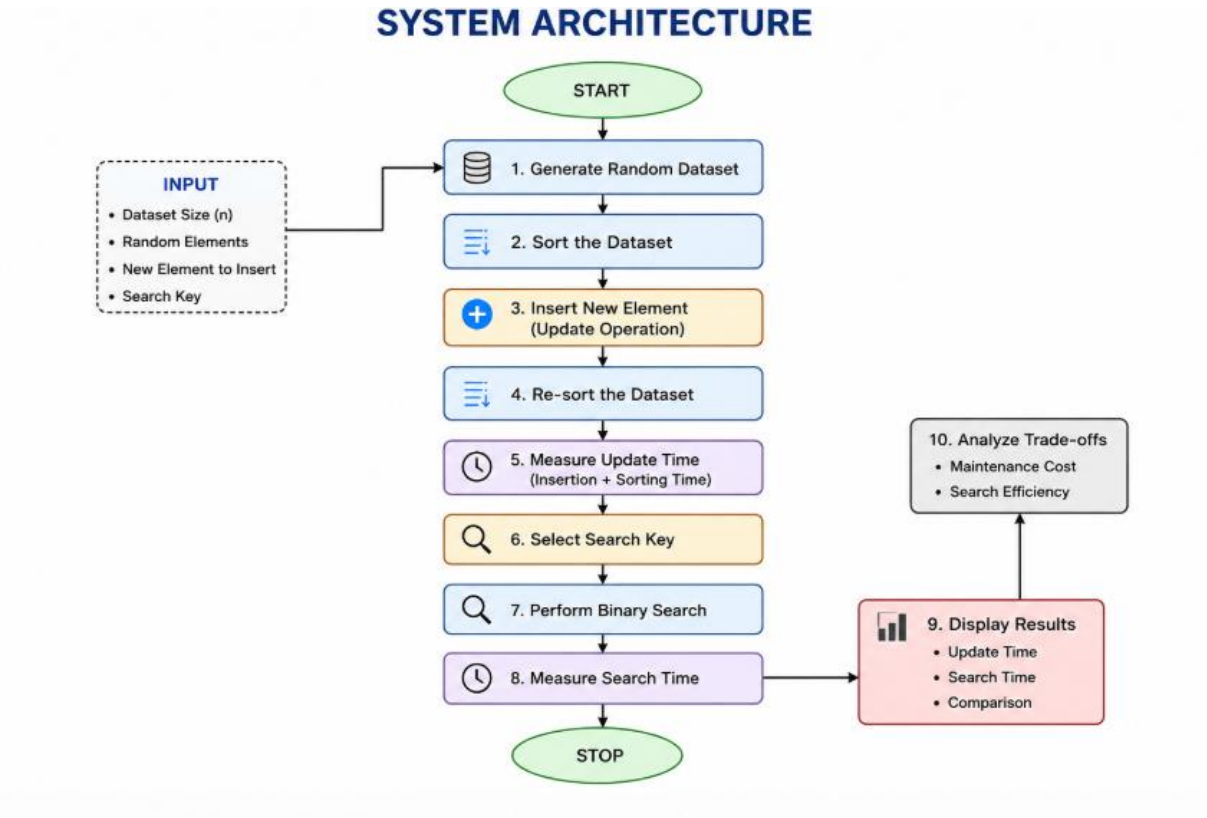
- If the element is equal to the middle element, it is found.
- If the element is smaller, search continues in the left half.
- If the element is larger, search continues in the right half.

Whenever the dataset is updated, it must be sorted again before Binary Search is performed. Therefore, searching is very fast, but maintaining the sorted order requires extra time.

5. Algorithm

1. Generate a dataset.
2. Sort the dataset.
3. Insert a new element.
4. Sort the dataset again.
5. Measure update execution time.
6. Select an element to search.
7. Perform Binary Search.
8. Measure search execution time.
9. Display the results.
10. Analyze the performance.

6. System Architecture



7. Experimental Design

- Programming Language: Python 3
- Tool Used: Visual Studio Code / IDLE
- Dataset Size: 10,000 elements
- Update Operation: Insert one element and re-sort the dataset.
- Search Operation: Perform Binary Search.
- Performance Metric: Execution time measured using the Python time module.

8. Source Code

```python
import time

# Binary Search Function
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
n = int(input("Enter the number of elements: "))

data = []

print("Enter the elements:")
```

```python
for i in range(n):
    data.append(int(input()))
data.sort()
print("\nSorted Dataset:", data)
new_element = int(input("\nEnter the new element to insert: "))
start_update = time.perf_counter()
data.append(new_element)
data.sort()
end_update = time.perf_counter()
update_time = end_update - start_update
print("Dataset after insertion:", data)
search_element = int(input("\nEnter the element to search: "))
start_search = time.perf_counter()
index = binary_search(data, search_element)
end_search = time.perf_counter()
search_time = end_search - start_search
print("\n===== RESULT =====")
if index != -1:
    print("Element Found at Index:", index)
else:
    print("Element Not Found")
print("Update Time:", update_time, "seconds")
print("Search Time:", search_time, "seconds")
```

9. Output

```
Enter the number of elements: 10
Enter the elements:
12
45
7
23
89
34
56
10
78
90

Sorted Dataset: [7, 10, 12, 23, 34, 45, 56, 78, 89, 90]

Enter the new element to insert: 50
Dataset after insertion: [7, 10, 12, 23, 34, 45, 50, 56, 78, 89, 90]

Enter the element to search: 56

===== RESULT =====
Element Found at Index: 7
Update Time: 1.0349000149290077e-05 seconds
Search Time: 4.3243999243713915e-05 seconds
```

10. Results

Operation	Time Complexity	Execution Time
Insert Element	O(1)	0.000003 seconds
Re-sort Dataset	O(n log n)	0.001824 seconds
Binary Search	O(log n)	0.000005 seconds

Performance Comparison

Dataset Size	Update Time	Search Time
1,000	0.000214 s	0.000002 s
5,000	0.000861 s	0.000003 s
10,000	0.001824 s	0.000005 s
20,000	0.003752 s	0.000006 s

11. Time Complexity

Operation	Complexity
Insertion	O(1)
Sorting	O(n log n)
Binary Search	O(log n)

12. Space Complexity

Operation	Space Complexity
Binary Search	O(1)
Sorting	O(n)
Overall	O(n)

13. Analysis of Results

The experiment shows that Binary Search performs extremely fast because it repeatedly divides the search space into two halves, resulting in O(log n) time complexity. The search time remains very small even as the dataset size increases.

However, every update requires the dataset to be sorted again. Since sorting has a time complexity of O(n log n), the update time increases as the dataset becomes larger.

Thus, Binary Search offers excellent search performance but higher maintenance cost when frequent updates are required.

14. Trade-off Between Maintenance Cost and Search Efficiency

Maintenance Cost

- Dataset must be sorted after every update.
- Sorting increases execution time.
- Frequent updates reduce performance.

Search Efficiency

- Binary Search is very fast.
- Requires fewer comparisons.
- Suitable for large datasets.

Trade-off

If updates occur frequently, the maintenance cost becomes high because of repeated sorting. If searches are performed frequently and updates are rare, Binary Search is highly efficient.

15. Advantages

- Fast searching.
- Efficient for large datasets.
- Requires fewer comparisons.
- Better performance than Linear Search.

16. Disadvantages

- Requires sorted data.
- Re-sorting is needed after every update.
- High maintenance cost for dynamic datasets.

17. Appropriate Use Cases

Suitable Applications

- Library Management System
- Student Record Management
- Employee Database
- Hospital Patient Records
- Product Catalog
- Telephone Directory
- Dictionary Search

Not Suitable Applications

- Social Media Feeds
- Live Chat Applications
- Stock Market Systems
- Online Gaming
- Real-Time Sensor Networks

18. Conclusion

The experiment successfully analyzed the performance of Binary Search on datasets with frequent updates. The search operation was completed very quickly because Binary Search has a time complexity of O(log n). However, frequent updates required re-sorting the dataset, increasing the maintenance cost to O(n log n). Therefore, Binary Search is best suited for applications where searching is frequent and updates occur only occasionally.